Lost & Found Portal: Implementation Plan (v3, 6 Weeks)

Stack: Next.js (web) | Django + DRF (API) | MySQL or PostgreSQL (schema ready for both) | Redis + Celery | Android app (Kotlin MVP)

Users: Students with `@iitrpr.ac.in` emails. **Roles:** Admin, User 1 (lost something), User 2 (found something).

Team assumption: 2 people. Every phase ends with a test gate: the next phase starts only when the gate passes.

How the portal works

- **User 1** submits a lost report: item, photos, hidden proof (marks only the owner knows), last seen place and time, and the reward amount they promise.
- **User 2** posts what they found (public post) and can also inform the admin, optionally leaving the item at the admin or security office.
- **Admin** reviews posts, matches lost and found, verifies the owner, supervises the handover and closes the case.
- **Survey-driven choice (32 responses):** handover is admin-verified, but the two parties choose to meet directly or via the office.

Data model: a case is a `posts` row (LOST or FOUND). A `matches` row links a lost post to a found post and moves through Suggested, Confirmed, Proof pending, Verified, Handed over, Closed (or Dismissed). See `schema_postgres.sql` and `schema_mysql.sql` (19 tables).

Architecture

- One versioned REST API (`/api/v1/`) for web and Android; old app versions keep working.
- Auth: JWT (simplejwt). Bearer tokens for Android, httpOnly cookies for web.
- Images: presigned direct uploads to S3 or Cloudinary, with thumbnails.
- Chat: REST + WebSocket (Django Channels), FCM push when the app is in the background.
- OpenAPI schema (drf-spectacular) so both clients get typed models.
- Django apps: `accounts`, `posts`, `matching`, `items` (QR), `chat`, `notifications`, `adminpanel`.

Week 1: Foundation, Sign-up and Roles (Phases 0, 1, 2)

Phase 0: Setup and Foundation

Build

- Repos `backend/` and `frontend/`, Docker compose (DB, Redis), `.env`, CI, common error format.
- Apply the schema as Django models and migrations; load seed categories and settings.
- Literature review table and unique angle (admin-verified handover, QR with no personal data, private contact).

Test

- Both apps boot, migrations run on a clean DB, CI is green.
- Seed data loads; schema constraints reject a bad email domain and a negative reward.

Phase 1: Authentication and Registration

Build

- Email must match `^[a-z0-9._%+-]+@iitrpr\.ac\.in$`, then OTP, then profile and password.
- OTP: 6 digits, stored hashed, 5 min expiry, 3 attempts, resend cooldown.
- Profile: full name, registration number, department, program and year, optional hostel, phone, photo.

- JWT access and refresh, logout blacklist, device registration, password reset through OTP.
- Email and registration number cannot be edited later; public views show only name and registration number.

Test

- pytest: wrong domain, expired OTP, lockout after 3 wrong OTPs, duplicate email or registration number, token refresh and logout.
- Playwright: register -> login end to end.

Phase 2: Roles and Permissions

Build

- Roles: Admin and User. User 1 and User 2 are behaviours decided by the post type.
- DRF permission classes; admin routes only under `/api/v1/admin/`; admins created by superuser command or invite.

Test

- A non-admin gets 403 on every admin route.
- A user cannot read or edit another user's private data.

Week 1 exit: a real student can register, log in and see a role-correct home page.

Week 2: Cases and Admin Portal (Phases 3, 4)

Phase 3: Lost Report to Admin (User 1)

Build

- Form: item name, category, description, photos, last seen place and time, hidden proof, reward amount.
- Submission creates a post in Pending state and notifies the admin.
- "My reports" page with status and admin remarks. Reward is a promised amount (honor-based in v1).
- Sensitive categories (wallet or cash) are never shown publicly.

Test

- Validation: required fields, image size and type limits, reward is a non-negative number.
- Post is Pending and visible only to its owner and admins.
- Playwright: submit form -> appears in My reports.

Phase 4: Admin Portal

Build

- Dashboard queue with filters (status, category, date); detail view with photos, hidden proof and history timeline.
- Actions: approve (publish), reject with reason, request more info, edit category, assign to another admin.
- Admin controls what is public and what stays private; audit log on every action.
- Hide or delete posts, ban or unban users, review abuse reports; statistics (registrations, cases per day, items returned, average time to return).

Test

- Approve publishes a sanitized post; reject notifies the user with the reason.
- Private proof never appears in any public API response.
- An audit log row exists for each admin action.

Week 2 exit: a lost report goes from submit to approved on the admin portal.

Week 3: Feed, Found Reports and Matching (Phase 5 + matching)

Phase 5: Public Feed and Found Reports (User 2)

Build

- Feed with category filters (bicycle, phone, laptop, earbuds, keys, ID card, bottle, umbrella, other), infinite scroll and search.
- "I found something" form; User 2 can publish a found post and/or inform the admin privately, with custody (with finder, admin office, security office).
- LOST and FOUND tags, reward badge, report button, auto-expiry after 30 days unless renewed.

Test

- Filters and pagination return correct results; the feed query uses the feed index (check with EXPLAIN).
- Found post appears on the feed; "Inform admin" creates an admin-only post.
- Playwright: filter by category, open a post.

Matching Engine

Build

- Celery task scores lost vs found pairs on category, date window, location keywords and description text (score 0 to 1).
- Pairs above the threshold (default 0.75) become Suggested matches in the admin match queue.

Test

- Unit tests with sample data: true pair scores above threshold, unrelated pair does not.
- Re-running the task creates no duplicate pair.

Week 3 exit: admin sees suggested matches for real test data.

Week 4: Verification, Handover and QR (Phases 6, 7)

Phase 6: Verification and Handover

Build

- Admin confirms a match, then asks hidden-proof questions (colour, scratches, wallpaper, serial number) and records each answer.
- Admin generates a one-time handover code (hash stored); owner shows it, admin marks Handed over.
- Meeting mode chosen by the parties: direct meet or via office.
- Reward record: amount settled and finder credited; post shows "Returned".

Test

- Wrong proof keeps the match at Proof pending; correct proof moves it to Verified.
- Handover code works once and then expires.
- Only admins can change match status.

Phase 7: QR System

Build

- Users register valuable items (bicycle, laptop, phone) and generate a QR with a random UUID link, printable as a sticker.
- Public scan page shows only the item and owner name; "Report found" sends the finder to the admin flow or private message.
- Search by name or registration number.

Test

- Public endpoint returns only whitelisted fields.
- Scan with several real phones and lighting conditions.
- Deactivated QR shows a neutral message.

Week 4 exit: a full case runs lost -> match -> verify -> handover -> closed.

Week 5: Chat and Android (Phases 8, 9)

Phase 8: Messaging and Notifications

Build

- Admin-mediated chat: users talk to the admin about their case; direct owner-finder chat only after admin approval.
- Notifications (in-app, email, FCM): approved or rejected, match found, info requested, handover ready, reward recorded.
- WebSocket live chat with a `since` cursor for offline sync; read receipts, unread counts, block and report.

Test

- Two users and an admin exchange messages live; an offline user gets a push and syncs on return.
- Blocked users cannot message; each notification fires once per event.

Phase 9: Android MVP

Build

- Kotlin + Jetpack Compose. Screens: login and OTP, feed, lost form, found form, My reports, QR scan (CameraX + ML Kit), chat, notifications.
- App Links so a scanned QR opens the app; app config endpoint for forced updates.

Test

- Contract tests against the OpenAPI schema.
- Run on low-end devices and slow campus Wi-Fi; Play Store internal testing track.

Week 5 exit: a student completes lost and found flows from an Android phone.

Week 6: Hardening and Launch (Phases 10, 11)

Phase 10: Hardening

Build

- DRF throttling, `django-axes`, input sanitisation, HTTPS only, image checks.
- Load test with Locust (about 200 concurrent users), index tuning, `python manage.py check --deploy`.

Test

- Security cases: XSS, SQL injection, IDOR on posts, matches and chats, admin privilege escalation. All pass.
- p95 API latency under 500 ms.

Phase 11: Deployment and Real Users

Deploy

- Frontend on Vercel; backend on Railway, Render or a campus server (Gunicorn + Nginx); managed DB with daily backups; Redis; S3 or Cloudinary; Sentry; uptime monitoring. Staging mirrors production.

Rollout

1. Internal test with the team and 1 to 2 admins for 2 to 3 days.
2. Beta in one hostel (20 to 30 students) for one week with a feedback form.
3. Fix issues, then campus-wide launch with posters and QR stickers.
4. Android release on Play Store.

Test: restore a backup into staging; run the full smoke test on production after each deploy.

Success metrics: registrations, posts per day, items returned, average time to return, admin response time, survey follow-up satisfaction.

Timeline Summary

Week	Work
1	Phases 0 to 2: setup, sign-up, roles
2	Phases 3 to 4: lost reports, admin portal
3	Phase 5 + matching: feed, found reports, match queue
4	Phases 6 to 7: verification, handover, QR
5	Phases 8 to 9: chat, notifications, Android MVP
6	Phases 10 to 11: hardening, deploy, beta

After Launch

- SLA timer: remind admin if a post is not reviewed in 24 hours.
- Campus map pin for location; helpful-finder badges.
- Image-similarity matching with an AI model.
- Weekly digest email; multi-campus support.

Risks

- Six weeks is tight for two people; Android is the first thing to trim to a smaller MVP.
- The MySQL schema file has not been executed yet; run it before choosing MySQL.
- Reward payment is honor-based in v1; real payments are a later phase.